Creative Software Design
6. Lab 1
Class

Dept. of Computer Science

Hanyang University

Classes

- A **class** is a user-defined data type
  - C++ **`class`** is similar to C **`struct`**
- Fine-grained access control on members
- Member variables as well as member functions
  - For types, accessing members by using scope resolution operator

```
int Point::getX();
```

  - For instances, accessing members by using . operator

```
Point p; p.getX();
```

  - For pointers, accessing members by using -> operator

```
Point* p; p->getX();
```

Instances

- Classes and instances are different
  - Classes ➔ data type
  - Instances ➔ data, object





- ***Instantiation***
  - Creating an instance of a class
  - Allocating memory store the instance
  - Multiple instances from a same class

```
class Point
{                          class
private:
  int x;
  int y;
public:
  void setX(int num) {x = num;}
  void setY(int num) {y = num;}
};

int main(void)             instances
{
  Point p1, p2;
  p1.setX(1); p1.setY(2);
  p2.setX(3); p2.setY(4);
  return 0;
}
```


Access Specifiers

keyword `private` identifies class members
that can be accessed only through the member functions of the class (data hiding)

keyword `class` identifies class definition

the class name becomes the name of this user-defined type

class members can be variables or functions

```
class Stock
{
private:
     char company[30];
     int shares;
     double share_val;
     double total_val;
     void set_tot() { total_val = shares * share_val; }
public:
     void acquire(const char * co, int n, double pr);
     void buy(int num, double price);
     void sell(int num, double price);
     void update(double price);
     void show();
};
```

keyword `public` identifies class members
that constitute the public interface for
the class (abstraction)

Member Functions

- Member functions defined in header files
  - Function definitions in classes
    - The functions will be ***inlined***
  - Function definitions outside the class but still in headers
    - `inline` qualifier makes the functions ***inline***
    - Same problem with declaring non-inline function in headers

```
class Stock {
private:
    ...
    void set_tot(){
        total_val = shares * share_val;
    }
public:
    ...
};
```

```
class Stock {
private:
    ...
    void set_tot();
public:
    ...
};

inline void Stock::set_tot(){
    total_val = shares * share_val;
}
```


#1 – Design Your Own Data Type

- Define a student type containing ID, name, and GPA
  - Use a `class` with direct member access
  - No constructor and destructor considered
  - No validation for ID and GPA considered
- Provide data into an instance of the type
  - example01.cpp : user input ➔ instances

```
$ ./example01
Register a student
ID: 2026012345
Name: Alice
GPA: 4.5
Registered students' information
{ID: 20262012345, Name: Alice, GPA: 4.5}
```

#2 – Member Functions - 1

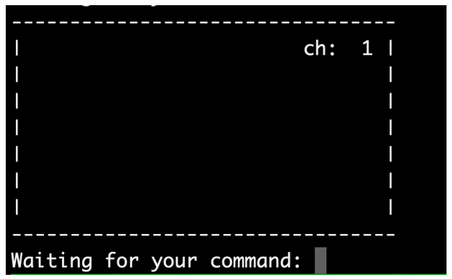
- Improve your example01
  - `ID` MUST be a 10-digit number
  - `GPA` MUST be a number between 0.0 and 4.5
  - For an invalid input, prompt the user to enter it again
  - `GetID()` shows the actual ID
  - `DataMask` can be used for hide a portion of an `ID` or `name`
    - `DataMask::GetStudentID(Student&);`
    - `DataMask::GetStudentName(Student&);`
- Getters and setters can be either inline or non-inline
  - Inline member functions are used for single-line definitions
  - Non-inline member functions are used for longer one

#3 – Member Functions - 2

- Define a class for MxN Matrix of an `int` number
  - Members are not allocated by dynamic memory allocation
    - MxN elements can be defined as an 1-D array
  - Member functions
    - `int GetCols(void)`
    - `int GetRows(void)`
    - `void SetElements(int* elements, int m, int n)`
    - `void Transpose(void)`
    - `void PrintElments()`

#4 – My Television

- A television object manages two states
  - Power: on and off
  - Channel: a number between 1 and 32
- User input: channel **u**p, channel **d**own, **i**n, **o**ut, **q**uit
- A display example



All Done!!

- Let's submit to ***gitlab***
- Push to ***gitlab*** after each lab session
  - At your VM instance (and make sure **pwd** at "week06_1")

```
# You are at {Your local project directory}/week06_1
$ git add *.cpp *.h Makefile
$ git commit –m "submission of 6-1"
$ git push origin main
```